

Assignment 3 — Bridge Pattern

Student: Nazym Kulchikova

Group: SE-2528

Topic: B — Notifications

Programming Language: Java

JDK: 17

1. Introduction

This project demonstrates the Bridge Design Pattern using a notification system. The main purpose of the Bridge Pattern is to separate an abstraction from its implementation so that both parts can be extended independently.

In this project, the abstraction represents different types of notifications, while the implementation represents different channels used to deliver notifications. This prevents the number of classes from growing unnecessarily when new notification types or delivery channels are added.

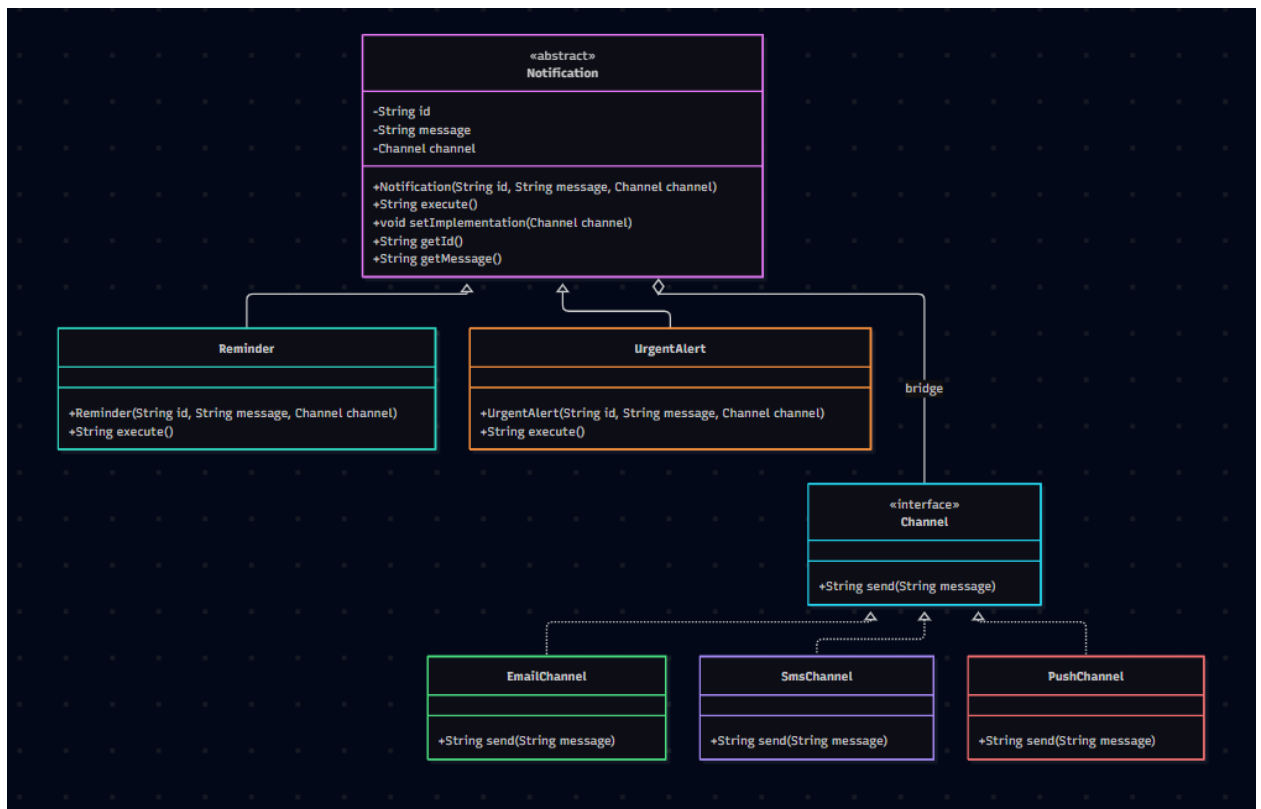
The project contains two independent hierarchies:

- Abstraction: Notification
- Concrete Abstractions: Reminder, UrgentAlert
- Implementor: Channel
- Concrete Implementors: EmailChannel, SmsChannel, PushChannel

2. Bridge Pattern Structure

The abstract class Notification is the main abstraction. It stores an interface-typed reference to Channel.

The abstraction hierarchy is:



This means that a **Reminder** or **UrgentAlert** can work with any implementation of the **Channel** interface.

3. Implementation

The **Channel** interface defines the operation used by all notification channels:

```
public interface Channel {
    String send(String message);
}
```

EmailChannel returns an email-style notification:

```
public class EmailChannel implements Channel {

    @Override
    public String send(String message) {
        return "EMAIL: [Envelope] " + message;
    }
}
```

SmsChannel returns a single-line SMS:

```
public class SmsChannel implements Channel {

    @Override
    public String send(String message) {
        return "SMS: " + message;
    }
}
```

PushChannel was later added as the third implementation:

```
public class PushChannel implements Channel {  
  
    @Override  
    public String send(String message) {  
        return "PUSH: [Notification] " + message;  
    }  
}
```

The abstract Notification class stores the bridge reference:

```
protected Channel channel;
```

The reference is supplied through the constructor:

```
public Notification(String id, String message, Channel channel) {  
    this.id = id;  
    this.message = message;  
    this.channel = channel;  
}
```

The abstraction therefore does not create concrete channel objects. It only works with the Channel interface.

4. Concrete Abstractions

Reminder represents a normal notification. Its execute() method sends the original message through the selected channel:

```
@Override  
public String execute() {  
    return channel.send(message);  
}
```

UrgentAlert represents an urgent notification. It adds the URGENT: marker before sending the message:

```
@Override  
public String execute() {  
    return channel.send("URGENT: " + message);  
}
```

This demonstrates that the abstraction hierarchy controls notification behavior, while the implementation hierarchy controls how the notification is delivered.

5. Runtime Implementation Switching

One important feature of the project is the setImplementation() method:

```
public void setImplementation(Channel channel) {  
    this.channel = channel;  
}
```

This method allows the implementation to be changed at runtime.

For example, the same Reminder object can first use an email channel:

EMAIL: [Envelope] Project deadline is tomorrow

and then switch to SMS:

SMS: Project deadline is tomorrow

The notification object itself does not need to be recreated.

T5 Same-Object Proof

Test T5 verifies that the same object is used before and after changing the channel.

The test checks:

- object identity using `==`;
- that the ID remains unchanged;
- that the message remains unchanged;
- that the output changes after switching the implementation.

The result is:

T5 PASS | sameObject=true | stateUnchanged=true | before=EMAIL: [Envelope] Project deadline is tomorrow | after=SMS: Project deadline is tomorrow

This proves that the implementation changes while the abstraction object and its domain data remain the same.

6. Five Important Code Elements

Code Excerpt 1 - Bridge Field

```
protected Channel channel;
```

The abstraction stores an interface-typed reference. This is the main connection between the two hierarchies.

Code Excerpt 2 — Constructor Injection

```
public Notification(String id, String message, Channel channel) {  
    this.id = id;  
    this.message = message;  
    this.channel = channel;  
}
```

The channel is supplied from outside. Notification does not create concrete implementations itself.

Code Excerpt 3 - Delegation

```
public String execute() {  
    return channel.send(message);  
}
```

The notification delegates the delivery operation to the Channel implementation.

Code Excerpt 4 - Runtime Switching

```
public void setImplementation(Channel channel) {  
    this.channel = channel;  
}
```

This allows the same notification object to switch between email, SMS, and push channels.

Code Excerpt 5 - Push Extension

```
public class PushChannel implements Channel {  
  
    @Override  
    public String send(String message) {  
        return "PUSH: [Notification] " + message;  
    }  
}
```

PushChannel demonstrates that a new implementation can be added without changing the existing notification hierarchy.

7. Bridge vs Adapter

Bridge and Adapter are both structural design patterns, but they solve different problems.

The main purpose of Bridge is to separate an abstraction from its implementation. It is useful when both dimensions of a system may need to vary independently.

Adapter has a different purpose. It makes an existing class with an incompatible interface work with the interface expected by the client.

In this project, Bridge is appropriate because notification types and notification channels are independent dimensions.

For example, without Bridge, the project could require classes such as:

- ReminderEmail
- ReminderSms
- ReminderPush
- UrgentAlertEmail
- UrgentAlertSms
- UrgentAlertPush

This approach would create many classes as the number of notification types and channels grows.

With Bridge, we can keep two separate hierarchies:

Notifications:

- Reminder
- UrgentAlert

Channels:

EmailChannel
SmsChannel
PushChannel

Therefore, adding a new channel does not require creating a separate class for every notification type.

8. Extension with PushChannel

The base version of the project contained:

- Channel
- EmailChannel
- SmsChannel
- Notification
- Reminder
- UrgentAlert

After the base commit, PushChannel was added as the third implementation.

Main.java was also updated with two additional tests:

- T6 - Reminder + PushChannel
- T7 - UrgentAlert + PushChannel

The existing abstraction classes and the existing Email and SMS implementations were not modified during the extension.

The base commit is:

324a9a0e234f4aef1a1c1a6af131bc847d884320

The extension commit is:

3e37816e24e42b04e8826a58afb49ae4a34ac93c

The extension.diff file contains the source changes between these commits.

9. Testing

The demo program contains seven tests.

T1 - Reminder + EmailChannel
T2 - Reminder + SmsChannel
T3 - UrgentAlert + EmailChannel
T4 - UrgentAlert + SmsChannel
T5 - Same-object implementation switch
T6 - Reminder + PushChannel
T7 - UrgentAlert + PushChannel

The final output is:

T1 PASS | Reminder + EmailChannel | result=EMAIL: [Envelope] Meeting at 10:00
T2 PASS | Reminder + SmsChannel | result=SMS: Meeting at 10:00
T3 PASS | UrgentAlert + EmailChannel | result=EMAIL: [Envelope] URGENT: Server is down

T4 PASS | UrgentAlert + SmsChannel | result=SMS: URGENT: Server is down
T5 PASS | sameObject=true | stateUnchanged=true | before=EMAIL: [Envelope] Project
deadline is tomorrow | after=SMS: Project deadline is tomorrow
T6 PASS | Reminder + PushChannel | result=PUSH: [Notification] Meeting at 10:00
T7 PASS | UrgentAlert + PushChannel | result=PUSH: [Notification] URGENT: Server is down
SUMMARY: 7/7 PASS

All seven tests pass successfully.

10. Design Trade-off

The main advantage of the Bridge Pattern in this project is flexibility. Notification types and delivery channels can be extended independently.

For example, a new PushChannel can be added without changing Reminder or UrgentAlert.

The main disadvantage is additional complexity. The project needs an interface, an abstract class, and several concrete classes instead of one simple class.

For a very small application, this additional structure may be unnecessary. However, when a system has several notification types and several delivery channels, Bridge provides better maintainability and reduces class multiplication.

11. Conclusion

The project demonstrates the Bridge Design Pattern through a notification system. The Notification hierarchy represents different types of notifications, while the Channel hierarchy represents different delivery mechanisms.

The two hierarchies are connected through composition. The implementation can be changed at runtime using `setImplementation()`, and T5 proves that the same notification object can use different channels without changing its state.

The addition of PushChannel demonstrates the extensibility of the design. The final demo contains seven tests and all of them pass.

12. Sources

1. Lecture 4 The Structural Design pattern – Bridge pattern. Senior Lecturer – Almas Ospanov